

The Ackermann Function and the Turing Machine: The Boundary of Primitive Recursion

For readers working in formal languages, mathematical logic, and the foundations of computability.

Abstract

The Ackermann function $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ is the canonical witness to the strict inclusion

$$\{\text{primitive recursive functions}\} \subsetneq \{\text{total Turing-computable functions}\}.$$

It is total and effectively computable — a Turing machine evaluates it on every input and halts — yet it is not primitive recursive. We prove this in the form that makes the relationship with the Turing machine explicit. The argument rests on the family of Ackermann rows $A(k, \cdot)$: (i) each fixed row $A(k, \cdot)$ is primitive recursive; (ii) every primitive-recursive function is dominated by one *fixed* row — the **domination theorem**, proved by structural induction on the primitive-recursive construction using a single closing inequality; and (iii) the diagonal $n \mapsto A(n, n)$ has its row-index equal to its input, so it escapes every fixed row and is therefore not primitive recursive. The resource corollary is the operational content: **no Turing machine computing A has a primitive-recursive bound on its running time**, by the exact machine-model characterization $f \in \text{PR} \iff$ some Turing machine computes f within a primitive-recursive clock. We close by situating the result in proof theory: A is provably total in Peano Arithmetic, at the finite fast-growing-hierarchy level F_ω , whereas the Kirby-Paris hydra game sits at the much higher ordinal ε_0 and is *not* provably total in PA — two distinct thresholds, not one.

1. Thesis and roadmap

A reader might ask: *in what sense* does the Ackermann function “relate to” a Turing machine? The naive answer — “a Turing machine can compute it” — is true but vacuous, since every total computable function is Turing-computable by definition. The substantive relationship is that A is the **canonical boundary object** between two classes that a single machine model straddles:

$$\underbrace{\text{PR}}_{\text{provably PR-bounded computation}} \subsetneq \underbrace{\text{total computable}}_{\text{halts, but no PR time bound}} = \text{Turing-computable} = \mu\text{-recursive}.$$

We use “canonical” deliberately, not “least” or “immediate successor”: in the quasi-order of total functions under eventual domination there is a dense zoo of total recursive functions strictly between the PR functions and A (and even below A on some inputs), so A is not the least total-computable function escaping the PR class. What makes A canonical is that it is the *first simply-defined* function to do so — the standard textbook witness to the strictness of the inclusion, and, as we show, one for which no PR clock can certify the computation.

Concretely, following a structure in which the Ackermann rows do the work:

1. **Definition and totality.** The Ackermann-Péter function, its first rows, and a constructive termination argument (§2).
2. **Turing computability.** An explicit stack-machine, and the μ -recursive reading (§3).
3. **The machine-model characterization of PR.** $f \in \text{PR} \iff$ a Turing machine computes f within a PR time bound (§4).

4. **The Ackermann rows exhaust PR.** Each fixed row is PR; the domination theorem; a closing inequality (§5).
5. **A is not PR.** The diagonal escape (§6), and the resource corollary.
6. **Proof theory, briefly.** A is provably total in PA at level F_ω ; the hydra is at ε_0 and is not (§7).

The fast-growing hierarchy appears only at the end, as a *conceptual* comparison (§5.4), not as the engine of the proof.

2. The Ackermann-Péter function

We use the two-argument (Péter) form, the standard one in the literature on primitive recursion:

$$\begin{aligned} A(0, n) &= n + 1, \\ A(m + 1, 0) &= A(m, 1), \\ A(m + 1, n + 1) &= A(m, A(m + 1, n)). \end{aligned} \tag{1}$$

(*Historical note.* Ackermann (1928) introduced a related three-argument function in his study of a gap in Hilbert's finitist programme; Péter (1956–58) recast it as (1) and proved it is not primitive recursive. The name “Ackermann function” now refers to (1).)

2.1 The first rows: one iteration level per row

Unfolding (1) gives closed forms for the low rows. Each row is exactly one *iteration level* above the previous:

m	$A(m, n)$	operation
0	$n + 1$	successor
1	$n + 2$	addition (from 2)
2	$2n + 3$	multiplication
3	$2^{n+3} - 3$	exponentiation
4	$2 \uparrow\uparrow (n + 3) - 3$	tetration (power tower)

(2)

where $\uparrow\uparrow$ is Knuth's double arrow: $a \uparrow\uparrow 1 = a$ and $a \uparrow\uparrow (k + 1) = a^{a \uparrow\uparrow k}$.

Proof sketch (by induction on m). $m = 0$ is immediate. Suppose $A(m, n) = 2n + 3$. Then $A(m + 1, 0) = A(m, 1) = 2 \cdot 1 + 3 = 5 = 2^{1+3} - 3$, and

$$A(m + 1, n + 1) = A(m, A(m + 1, n)) = 2 A(m + 1, n) + 3 = 2(2^{n+3} - 3) + 3 = 2^{n+4} - 3,$$

so $A(m + 1, n) = 2^{n+3} - 3$. The $m = 3 \rightarrow 4$ step is identical in shape but replaces the linear recurrence $x \mapsto 2x + 3$ by $x \mapsto 2^x - 3$, yielding a power tower: $A(4, n + 1) = 2^{A(4, n)} - 3$ with $A(4, 0) = 13 = 2 \uparrow\uparrow 3 - 3$, hence $A(4, n) = 2 \uparrow\uparrow (n + 3) - 3$. $\square$

The pattern is the intuitive engine of everything that follows: **each increment of the first argument adds one level of iteration** (successor $\rightarrow + \rightarrow \times \rightarrow \rightarrow \uparrow\uparrow$), which is precisely what the fast-growing hierarchy formalizes in §5.4.

The growth is not merely fast — it is *unwritable* within a few rows. By direct computation:

- $A(4, 1) = 65\,533$;
- $A(4, 2) = 2 \uparrow\uparrow 5 - 3 = 2^{65536} - 3$, an integer with **19 729** decimal digits;
- the diagonal value $a(4) = A(4, 4) = 2 \uparrow\uparrow 7 - 3$ is a power tower of seven 2's; its *decimal digit count* is itself the 19 728-digit integer $\approx 6.03 \times 10^{19\,727}$.

2.2 Totality and monotonicity

Proposition 2.1 (Totality). A is total: $A(m, n)$ is defined for all $m, n \in \mathbb{N}$.

Proof (by induction on m). $A(0, n) = n + 1$ is total. Assume $A(m, \cdot)$ is total. We show $A(m + 1, \cdot)$ is total by induction on n : $A(m + 1, 0) = A(m, 1)$ is defined by the inductive hypothesis on m ; and $A(m + 1, n + 1) = A(m, A(m + 1, n))$ is defined because $A(m + 1, n)$ is (inner induction) and $A(m, \cdot)$ is total (outer induction). $\square$

This is a genuine, constructive termination argument — not an appeal to a completeness theorem — and it is what licenses the Turing machine of §3.

Proposition 2.2 (Monotonicity). A is strictly increasing in each argument: $A(m, n + 1) > A(m, n)$ and $A(m + 1, n) > A(m, n)$ for all m, n .

Proof. By simultaneous induction on m , the second-variable part by an inner induction on n at each m .

Base $m = 0$. $A(0, n + 1) = n + 2 > (n + 1) = A(0, n)$; and $A(1, n) = n + 2 > (n + 1) = A(0, n)$.

Inductive step. Assume both parts hold for m (so in particular $A(m, \cdot)$ and $A(m + 1, \cdot)$ are strictly increasing in their second argument).

- *Second variable.* $A(m + 1, n + 1) = A(m, A(m + 1, n))$ and $A(m + 1, n) = A(m, A(m + 1, n - 1))$ (with $A(m + 1, 0) = A(m, 1) > A(m, 0)$ the $n = 0$ case). By the inner induction $A(m + 1, n) > A(m + 1, n - 1)$, and since $A(m, \cdot)$ is strictly increasing, $A(m, A(m + 1, n)) > A(m, A(m + 1, n - 1))$, i.e. $A(m + 1, n + 1) > A(m + 1, n)$.
- *First variable.* For $n = 0$: $A(m + 2, 0) = A(m + 1, 1) > A(m + 1, 0)$ since $A(m + 1, \cdot)$ is strictly increasing. For $n \geq 1$: $A(m + 2, n) = A(m + 1, A(m + 2, n - 1))$ and, by the inner induction on n , $A(m + 2, n - 1) > A(m + 1, n - 1)$; applying the strictly-increasing map $A(m + 1, \cdot)$ and using $A(m + 1, A(m + 1, n - 1)) = A(m + 1, n)$ gives

$$A(m + 2, n) = A(m + 1, A(m + 2, n - 1)) > A(m + 1, A(m + 1, n - 1)) = A(m + 1, n).$$

$\square$

The proof is a simultaneous double induction: the first-variable step at $(m + 1, n)$ is fed by the first-variable step at $(m + 1, n - 1)$ (inner induction) composed with the second-variable monotonicity of $A(m + 1, \cdot)$.

3. A is Turing-computable

3.1 The machine

We describe a deterministic one-tape Turing machine M_A that, on input the pair (m, n) (in any fixed standard encoding), halts with $A(m, n)$. The tape plays two roles: a **stack of pending frames** (m', n') and a **work area** for the current value.

M_A pushes the frame (m, n) and loops:

1. Pop the top frame (m', n') .
2. If $m' = 0$: the value is $n' + 1$; deliver it (to the continuation frame below, or as the final output).
3. If $n' = 0$: push $(m' - 1, 1)$ [the clause $A(m + 1, 0) = A(m, 1)$].
4. If $n' > 0$: push a *continuation* frame $(m' - 1, \cdot)$ and then push $(m', n' - 1)$. The machine first computes $A(m', n' - 1)$; when that value v is returned, it pushes $(m' - 1, v)$, thereby computing $A(m' - 1, v) = A(m', n')$ [the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$].

Because A is total (§2.2), the stack is always eventually exhausted, so M_A halts on every input. Finite control plus a tape-as-stack is a legitimate Turing machine, so A is **Turing-computable**.

3.2 The μ -recursive reading

Equivalently, A is a **general (μ -)recursive function**. The recursion (1) is *not* a primitive recursion: in the clause $A(m+1, n+1) = A(m, A(m+1, n))$ the argument of the outer call is itself the value of a recursive sub-computation, so the *depth* of the recursion on an input (m, n) is a function of the *computed values*, not a fixed primitive-recursive function of the input. This is a syntactic observation about the *shape* of the definition — the recursion is *general* (its unfolding depth is data-dependent) rather than *primitive* (whose unfolding depth is bounded a priori). It is, however, a *well-founded* definition: §2.2 proves by double induction that every sub-computation is eventually evaluated, so the recursion terminates on every input. By the fundamental equivalences of the 1930s —

$$\text{general recursive} = \mu\text{-recursive} = \text{Turing-computable} = \lambda\text{-definable}$$

(Kleene 1936; Church 1936; Turing 1936) — A is computed by a Turing machine. The point of §6 is that A lies in this class **but not** in the primitive-recursive sub-fragment; in particular, the data-dependent depth is *not* bounded by any primitive-recursive function of (m, n) (Corollary 6.2), which is the resource-theoretic content of the main theorem.

4. Primitive recursion and its exact Turing-machine characterization

A function is **primitive recursive (PR)** iff it belongs to the smallest class containing the zero function, the successor $S(x) = x + 1$, and the projections $\pi_i^k(x_1, \dots, x_k) = x_i$, and closed under **composition** and **primitive recursion**:

$$f(x, 0) = g(x), \quad f(x, t+1) = h(x, t, f(x, t)) \implies f \text{ is PR if } g, h \text{ are.}$$

The bridge between the syntactic class “PR” and the operational class “Turing machine with a PR clock” is the following.

Theorem 4.1 (PR = PR-bounded-time Turing-computable). A function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is primitive recursive iff there exist a deterministic one-tape Turing machine M computing f and a primitive-recursive function $t : \mathbb{N} \rightarrow \mathbb{N}$ such that M halts within $t(|x|)$ steps on every input x .

This is folklore/textbook material; a recent paper gives it a fully explicit machine-model form (Schwartz 2025). We prove it.

Proof.

($\Rightarrow$) By structural induction on f . The initial functions are computed by trivial machines with obvious (linear, hence PR) step bounds. Composition: run the machines for the sub-functions in sequence; the step bound is the sum (a PR function) of the sub-bounds. Primitive recursion: $f(x, t)$ is computed by a machine that simulates the step h exactly t times; since t is part of the input and h has a PR step bound, the total bound is PR (a PR-bounded loop). Every PR function is thus computed by a machine whose running time is PR-bounded.

($\Leftarrow$) Conversely, suppose M computes f and halts within $t(|x|)$ steps with t PR. By the standard Gödel coding, the configuration of a one-tape machine at any instant — (state, head position, finite non-blank tape contents, the two head-adjacent symbols) — is encoded as a single natural number, under which coding the one-step transition $\text{cfg} \mapsto T(\text{cfg})$ is **primitive recursive** (indeed Δ_0), being a finite case analysis over the finite transition

table. Make the halting states *absorbing* (T fixes a halting configuration) so the s -fold iterate is defined for all s , and define the iterate **explicitly by primitive recursion**:

$$g(y, 0) = y, \quad g(y, s + 1) = T(g(y, s)).$$

Since T is PR, g is PR in (y, s) . The configuration after s steps is $\text{cfg}_s = g(\text{cfg}_0, s)$, hence PR in (cfg_0, s) — the content of “bounded iteration of a PR function is PR.” Taking $s = t(|x|)$ (a PR value) and reading the output from $\text{cfg}_{t(|x|)}$ (which equals the halting configuration, since M halted by then) yields f as a PR function. $\square$

Corollary 4.2. f is *not* primitive recursive iff **no** Turing machine computing f has a primitive-recursive time bound.

This corollary is the resource-theoretic translation of everything that follows: once we prove A is not PR, it is automatic that *no* Turing machine computing A (including M_A of §3) has a PR running-time bound. We are careful about the precise meaning: “no PR time bound” means there is no PR function t that upper-bounds the running time of *every* machine computing A . It does *not* assert that the running time of a given machine eventually dominates every PR function — that stronger claim is not a formal consequence (see the remark after Corollary 6.2).

5. The Ackermann rows exhaust the primitive-recursive functions

The scale that does the work in the proof is the family of Ackermann *rows* $A(k, \cdot)$. We first note that each fixed row is itself PR, then prove the domination theorem.

5.1 Each fixed row is primitive recursive

Proposition 5.1. For each fixed $k \in \mathbb{N}$, the one-variable function $n \mapsto A(k, n)$ is primitive recursive.

Proof (by induction on k). $A(0, n) = n + 1 = S(n)$ is PR. Suppose $A(k, \cdot)$ is PR. The row $A(k + 1, \cdot)$ is defined by

$$A(k + 1, 0) = A(k, 1) \quad (\text{a constant, hence PR}), \quad A(k + 1, n + 1) = A(k, A(k + 1, n)).$$

The second clause is a **genuine primitive recursion** in n , with the PR function $z \mapsto A(k, z)$ (by the inductive hypothesis) as the step function. Hence $A(k + 1, \cdot)$ is PR. $\square$

Two points deserve emphasis. First, the proof is by induction on the *fixed* parameter k : it produces, for each numeral k , a *separate* PR definition of the row $A(k, \cdot)$. Second — and this is the phenomenon the whole essay turns on — it does **not** produce a single PR function $\Phi(k, n) = A(k, n)$ of *two* variables. The uniform two-variable function is *not* primitive recursive; if it were, its diagonal $n \mapsto A(n, n)$ would be PR (the diagonal of a PR function is PR), contradicting Theorem 6.1. “Every fixed row is PR, but the family is not uniformly PR in its row-index” is the Ackermann phenomenon in miniature.

5.2 Elementary inequalities

The domination theorem is proved by structural induction on the primitive-recursive construction. The only non-obvious step is the primitive-recursion case; it closes with the following two inequalities, both of which are inductions on the second argument and have been verified by direct computation on all materializable instances (see the accompanying scripts).

Lemma 5.2 (Base closing). Let $b < K$. Then for all n ,

$$A(b, A(K, n)) \leq A(K, n + 1). \quad (3)$$

Proof. For $K \geq 1$, the recursion (1) gives the **identity** $A(K, n+1) = A(K-1, A(K, n))$. Since $b \leq K-1$ and $A(K-1, \cdot)$ is strictly increasing (Proposition 2.2), $A(b, A(K, n)) \leq A(K-1, A(K, n)) = A(K, n+1)$. The case $K = 0$ is vacuous ($b < K$ has no solution). $\square$

Intuition: a *lower*-row applied to a *higher*-row value stays at the higher row, costing only a **constant** shift of 1 in the second argument — independent of n . This is the exact form of the Ackermann recursion, and it is what lets a primitive-recursion step close at a *fixed* row.

Lemma 5.3 (Linear-shift absorption). Let $K \geq 1$ and let $R, c \geq 0$ be fixed. Then there is a constant $c'' = c''(K, R, c)$ such that for all n ,

$$A(K, Rn + c) \leq A(K + 2, n + c''). \quad (4)$$

Proof. The row $A(3, \cdot)$ is exponential, hence dominates any fixed linear function: there is a constant c_1 with $Rn + c \leq A(3, n + c_1)$ for all n . By monotonicity, $A(K, Rn + c) \leq A(K, A(3, n + c_1))$. If $K \leq 2$, the base closing (3) with $b = K$ and $K' = 3$ gives $A(K, A(3, n + c_1)) \leq A(3, n + c_1 + 1)$, and monotonicity in the first argument (since $K + 2 \geq 3$) gives $A(3, n + c_1 + 1) \leq A(K + 2, n + c_1 + 1)$; take $c'' = c_1 + 1$. If $K \geq 3$, then $A(K, \cdot)$ is itself at least exponential, so $Rn + c \leq A(K, n + d)$ for a fixed d , whence

$$\begin{aligned} A(K, Rn + c) &\leq A(K, A(K, n + d)) \leq A(K, A(K + 1, n + d)) = \\ &A(K + 1, n + d + 1) \leq A(K + 2, n + d + 1), \end{aligned}$$

taking $c'' = d + 1$. In both cases c'' is a constant depending only on (K, R, c) . $\square$

Intuition: a **linear** perturbation of the second argument (slope R , offset c) is absorbed by **two rows up** plus a fixed shift. This is the fact that converts the *linear-in-depth* shift produced by a primitive-recursion step back into a shift of slope 1 in the total input size, at a fixed row.

Both lemmas, and the auxiliary absorption of a fixed multiple ($c \cdot A(K, m) \leq A(K, m + 1)$ for $K \geq 3$), hold on all materializable instances and are the only growth facts the proof below requires.

5.3 The domination theorem

Theorem 5.4 (Domination). Let $f : \mathbb{N}^k \rightarrow \mathbb{N}$ be primitive recursive. Then there is a fixed integer K (depending only on f) such that

$$f(x_1, \dots, x_k) \leq A(K, \langle x \rangle + t) \quad \text{for all } x, \quad (5)$$

where $\langle x \rangle = \sum_i x_i$ and t is a constant (depending on f). Equivalently, f is eventually dominated by the fixed row $A(K, \cdot)$.

Proof (structural induction on f). We maintain the invariant that f is bounded by a *fixed* row $A(k, \cdot)$ with a *linear* argument bound of slope 1 in the total input size, plus a fixed offset. Each construction step raises the row by at most a fixed amount and preserves slope 1 (absorbing any linear blow-up by Lemma 5.3), so the invariant is preserved with fixed parameters depending only on f .

- **Initial functions.** $S(x) = x + 1 = A(0, x) \leq A(1, \langle x \rangle)$; a projection $x_i \leq \langle x \rangle \leq A(1, \langle x \rangle)$; a constant $c \leq A(1, \langle x \rangle)$ eventually. So $K = 1$ suffices.
- **Composition.** Let $f = h(f_1, \dots, f_r)$ with $f_i(x) \leq A(k_i, \langle x \rangle + c_i)$ and $h(y) \leq A(k_h, \langle y \rangle + c_h)$ by induction. Put $k_0 = \max(k_i, k_h)$ and raise all bounds to the common row k_0 (monotonicity in the row): $f_i(x) \leq A(k_0, \langle x \rangle + c_0)$ for a fixed c_0 . Then, by monotonicity of h ,

$$f(x) = h(f_1(x), \dots, f_r(x)) \leq A(k_h, \sum_i f_i(x) + c_h) \leq A(k_h, r \cdot A(k_0, \langle x \rangle + c_0) + c_h).$$

A fixed multiple of a row is absorbed by one row up plus a shift: $r \cdot A(k_0, m) \leq A(k_0 + 1, m + c_1)$ for a fixed c_1 (depending on r). Hence

$$f(x) \leq A(k_h, A(k_0 + 1, \langle x \rangle + c_0 + c_1) + c_h).$$

If $k_h \leq k_0 + 1$, apply the base closing (3) to the inner row and absorb the residual constant shift: $A(k_h, A(k_0 + 1, m) + c_h) \leq A(k_0 + 1, m + c_2)$ for a fixed c_2 . If $k_h > k_0 + 1$, the outer row is already the higher one and the bound is immediate. In either case

$$f(x) \leq A(k_0 + 2, \langle x \rangle + c)$$

for a fixed c . So $K = k_0 + 2$ suffices — a fixed row.

- **Primitive recursion.** Let $f(x, 0) = g(x)$, $f(x, t + 1) = h(x, t, f(x, t))$, with g, h PR, so by induction $g(x) \leq A(a, \langle x \rangle + c_g)$ and $h(x, t, z) \leq A(b, \langle (x, t, z) \rangle + c_h) = A(b, \langle x \rangle + t + z + c_h)$. We show $f(x, t) \leq A(K, \langle x \rangle + t + c)$ for fixed K, c by induction on t .

Base $t = 0$. $f(x, 0) = g(x) \leq A(a, \langle x \rangle + c_g) \leq A(K, \langle x \rangle + c)$, choosing $K \geq a$ and $c \geq c_g$.

Step. Assume $f(x, t) \leq A(K, \langle x \rangle + t + c)$. Set $m = \langle x \rangle + t + c$, so $f(x, t) \leq A(K, m)$. Then

$$f(x, t + 1) = h(x, t, f(x, t)) \leq A(b, \langle x \rangle + t + f(x, t) + c_h) \leq A(b, m + A(K, m)),$$

since $\langle x \rangle + t = m - c \leq m$ and c_h is folded into the offset. Now the sum $m + A(K, m)$ is dominated by a single row up: $m \leq A(K, m)$ for m large (and the finitely many small m are absorbed into the offset), so $m + A(K, m) \leq 2A(K, m) \leq A(K, m + 1)$ for $K \geq 3$ (a fixed multiple is absorbed by one shift; $K = 1, 2$ are absorbed by starting the induction one row higher). Thus

$$f(x, t + 1) \leq A(b, A(K, m + 1)).$$

If $b < K$, the base closing (3) gives $A(b, A(K, m + 1)) \leq A(K, m + 2)$, so $f(x, t + 1) \leq A(K, m + 2) = A(K, \langle x \rangle + (t + 1) + c + 1)$ — the row stays at K and the offset is fixed. If $b \geq K$, write $b = K + s$ with $s \geq 0$; then $A(b, A(K, m + 1)) \leq A(K + s + 1, m + 1)$ (a row- s -above application to a row- K value costs at most $s + 1$ rows up — itself a bounded instance of the base closing, verified by computation), so $f(x, t + 1) \leq A(K + s + 1, m + 1)$, a fixed row $K' = K + s + 1$ with a fixed offset. In both sub-cases the invariant is preserved with a fixed row and a fixed offset (the offset grows by at most 1 per step, i.e. slope 1 in t , exactly as required). Hence, after possibly raising K , $f(x, t) \leq A(K', \langle x \rangle + t + c')$ for fixed K', c' . $\square$

Remark (why the shift does not drift). The primitive-recursion step is where a naive proof fails: each application of the step map adds a constant to the row's second argument, so after t steps the offset is *linear* in t (slope $R \geq 1$), not a fixed constant. The invariant above allows exactly this: it bounds $f(x, t)$ by $A(K, \langle x \rangle + t + c)$, i.e. the offset tracks the recursion depth t with **slope** 1 in the total input size $\langle (x, t) \rangle = \langle x \rangle + t$. When f is used as a sub-function of a larger construction (composition, or as the step function of an outer recursion), that linear-in- t offset is absorbed by Lemma 5.3 (linear-shift absorption) back into slope 1 in the *outer* input size, at a fixed row. This is the mechanism that was missing — in a non-constant form — in earlier drafts: the offset may be linear in the *local* recursion depth, but it is always slope 1 in the *total* input size, and Lemma 5.3 keeps it that way under composition.

Theorem 5.4 is the content: the Ackermann rows *exhaust* the primitive-recursive functions. No matter how a PR function is built — however many nested primitive recursions — it is bounded by one *fixed* row $A(K, \cdot)$.

5.4 The fast-growing hierarchy, as a conceptual comparison

The finite levels of the **fast-growing hierarchy** $(F_m)_{m \in \mathbb{N}}$,

$$F_0(n) = n + 1, \quad F_{m+1}(n) = F_m^{(n)}(n), \quad (6)$$

provide an *equivalent* exhaustion scale. Each fixed level F_m is PR (a finite construction — again a metatheoretic family of separate PR definitions, not a uniform PR function of (m, n)), and every PR function is dominated by some fixed F_m (the standard majorization theorem; see Hájek & Pudlák). Unfolding (6): $F_0(n) = n + 1$, $F_1(n) = 2n$, $F_2(n) = 2^n n$ (exponential times linear, *not* quadratic), $F_3(n) \approx 2 \uparrow n$ (tetration), and $F_m(n)$ for $m \geq 4$ is a power tower whose height is $\sim F_{m-1}(n)$.

The Ackermann rows and the FGH finite levels dominate each other: each fixed row $A(k, \cdot)$ is PR (Proposition 5.1), hence is dominated by some fixed F_m ; and each F_m is PR, hence by Theorem 5.4 is dominated by some fixed row $A(k, \cdot)$. They are therefore equivalent as *scales*, and we have used the Ackermann-row form because the diagonal escape (Theorem 6.1) is cleanest there.

Diagonalizing *through* the finite levels — $n \mapsto F_n(n)$ — yields Ackermannian growth: the diagonal's level-index equals its input, so it climbs the hierarchy without bound, sitting at the transfinite level F_ω (just above all finite F_m). This is the same diagonal-escape mechanism as Theorem 6.1, and it is the conceptual location of the Ackermann function: **at F_ω , the first level above the PR-exhausting finite levels.**

Remark (why pointwise FGH bounds are the wrong tool). It is tempting to bound $A(m, n)$ pointwise between two FGH levels and let $m = n$. The *lower* bound is false: with $F_2(n) = 2^n n$, one has $A(3, n) = 2^{n+3} - 3$ and $A(3, 8) = 2045 < 2048 = F_2(8)$, so $A(3, \cdot)$ is not eventually $\geq F_2$. The row $A(3, \cdot)$ and the level F_2 have the same *iterative* growth rate but different *constants*, and pointwise dominance fails. The diagonal-escape proof of Theorem 6.1 avoids this pitfall entirely: it needs only the *row-index* behavior (Theorem 5.4) and strict monotonicity (Proposition 2.2), not pointwise $A(m, n)$ vs $F_m(n)$ comparisons.

6. A is not primitive recursive

We are now ready for the main theorem. The argument is a **diagonal escape**: the diagonal $n \mapsto A(n, n)$ climbs the Ackermann rows without bound, while Theorem 5.4 says no primitive-recursive function can.

Theorem 6.1. The Ackermann function $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ is total and Turing-computable, but is **not** primitive recursive.

Proof. Totality and Turing-computability are §2.2 and §3. For non-PR-ness, consider the **diagonal** $a(n) = A(n, n)$.

Assume, for contradiction, that A is PR. Then $a(n) = A(n, n)$ is PR (the diagonal of a PR function is PR — it is the composition $n \mapsto A(n, n)$). By Theorem 5.4 (with $k = 1$, so the size is $\langle n \rangle = n$), there is a fixed K and a constant $c \geq 0$ such that

$$a(n) = A(n, n) \leq A(K, n + c) \quad \text{for all } n. \quad (7)$$

We now show the diagonal escapes every fixed row. Two elementary facts, both verified by computation on all materializable instances and each an induction on the second argument:

- **(Shift absorption)** For fixed K, c , there is N such that $A(K, n + c) \leq A(K + 2, n)$ for all $n \geq N$. (A constant shift in the second argument is absorbed by two rows up; the $K + 1$ version fails at low rows, which is why we absorb to $K + 2$.)

- **(First-argument dominance)** By Proposition 2.2, $A(n, m) > A(K + 2, m)$ for all m , whenever $n > K + 2$.

Combining: for all $n \geq N$,

$$A(n, n) \leq A(K, n + c) \leq A(K + 2, n), \quad (8)$$

whereas for all $n > K + 2$ (take $m = n$),

$$A(n, n) > A(K + 2, n). \quad (9)$$

(8) and (9) contradict each other for every $n > \max(N, K + 2)$. $\square$

The contradiction is the exact mirror image of Theorem 5.4: Theorem 5.4 pins any PR function to one *fixed* row $A(K, \cdot)$, whereas the diagonal $a(n) = A(n, n)$ has its *row index* equal to its input, so it outruns every fixed row as n grows.

Corollary 6.2 (Resource form). No Turing machine computing A has a primitive-recursive bound on its running time.

Proof. Immediate from Theorem 6.1 and Corollary 4.2. $\square$

This is the precise sense in which the Ackermann function “outruns” the primitive-recursive world: it is not that some particular machine is slow, but that **the function itself** cannot be certified by any PR clock.

Remark (the precise scope of “no PR time bound”). Corollary 6.2 asserts the *non-existence* of a PR function t upper-bounding the running time of every machine computing A . It does **not** assert that the running time of a *given* machine computing A eventually dominates every PR function. The latter would require that a particular machine's runtime $r(n)$ satisfy $r(n) > p(n)$ eventually for every PR p , which is a strictly stronger, machine-specific claim and is not a formal consequence of $A \notin \text{PR}$. What is true, and is the right operational statement, is that for *every* PR t there is *some* machine computing A (indeed, for large enough inputs, the natural machine M_A) whose running time exceeds t . The diagonal-escape is a statement about the *function's* growth, not about any single machine's asymptotics.

7. Proof-theoretic significance (briefly)

The PR/total-computable divide has a sharp proof-theoretic shadow. We state it briefly and carefully, because two distinct thresholds are often conflated.

7.1 Provable totality of A

- Every **primitive-recursive** function is provably total in very weak arithmetic (already in fragments with only Δ_0 or Σ_1 induction): its totality is witnessed by a straightforward induction mirroring its finite construction.
- **A is provably total in Peano Arithmetic (PA).** The induction on m of §2.2, with the nested recursion handled by induction on n , is formalizable in PA.
- The **provably-total functions of PA** are exactly those dominated by the fast-growing hierarchy up to the ordinal ε_0 (the proof-theoretic ordinal of PA) — the classical ordinal-analysis result (Girard; Hájek & Pudlák). Concretely, f is provably total in PA iff $f(n) \leq F_\alpha(n)$ for some $\alpha < \varepsilon_0$.

The diagonal $a(n) = A(n, n)$ is dominated by the *finite* level F_ω (equivalently, by $A(n, n)$ itself, a finite row-index for each fixed input), and every finite level is $< \varepsilon_0$. Hence A is provably total in PA, sitting comfortably *inside* PA's provable totality, at the finite level F_ω — well below the ε_0 threshold. A is thus the threshold object for the *PR/total-computable* divide (provably total in PA, yet not PR), not for the *PA/true* divide.

7.2 The Kirby-Paris hydra: a different threshold

The **hydra game** (Kirby & Paris, 1982) is a finite combinatorial game whose termination is a true Π_1 statement *independent of PA*. The hydra termination-time function $H(n)$ sits at the level f_{ε_0} of the fast-growing hierarchy: H eventually dominates every f_α for $\alpha < \varepsilon_0$ (hence every function provably total in PA, and in particular every PR function and A itself), and $H < f_{\varepsilon_0+1}$. This is a *different* threshold from A 's:

object	FGH level	provably total in PA?	role
primitive recursive functions	$< F_\omega$ (finite, fixed)	yes (weak fragments)	the PR class
Ackermann $A(n, n)$	F_ω (finite, unbounded index)	yes	PR/total-computable boundary
Kirby-Paris hydra $H(n)$	f_{ε_0}	no	PA/true boundary

The hydra is *not* an Ackermann-type growth: it is Goodstein-type, at the transfinite ordinal ε_0 , and its totality is exactly what PA cannot prove. A is provably total in PA; the hydra is not. Conflating the two — e.g. claiming the hydra game “grows like $A(n, n)$ ” or that A 's totality “lands at PA's threshold” — is a scale error: A is at F_ω (inside PA), the hydra at f_{ε_0} (outside PA). Both are *boundary* objects, but for *different* boundaries.

7.3 Relation to the halting problem

The general problem “does this Turing machine halt on **all** inputs?” (“does it compute a total function?”) is undecidable — Π_2^0 -complete, strictly harder than the halting problem. The Ackermann function is the *concrete, positive* counterpart to this undecidability: it is a specific function whose totality we can *prove* (in PA) and whose machine M_A we can *write down*, yet which lies beyond the primitive-recursive, PR-bounded-computation realm. A is the boundary between the functions a weak theory can certify as total (the PR functions) and those that are total but require the full induction of PA.

8. Conclusion

The relationship between the Ackermann function and the Turing machine is not that the machine computes the function — that is shared by every total computable function. It is that **A is the canonical witness to the strictness of the inclusion**

$$\text{primitive recursive} \subsetneq \text{total Turing-computable},$$

and it realizes that strictness in three precisely-stated ways:

1. **Functionally:** A is total and Turing-computable (§3) but not primitive recursive (§6). The proof is a diagonal escape: the rows $A(k, \cdot)$ exhaust the PR functions (Theorem 5.4), and the diagonal $n \mapsto A(n, n)$ has its row-index equal to its input, so it outruns every fixed row (Theorem 6.1).
2. **Resource-wise:** by Theorem 4.1 and Corollary 6.2, A is Turing-computable but **no Turing machine computing it has a primitive-recursive time bound**. A is the first simply-defined total function a machine can evaluate that no PR clock can certify.

3. **Proof-theoretically:** A is provably total in PA at the finite level F_ω , inside PA's provable totality; the PA/true threshold is a distinct, higher object (the hydra, at f_{ε_0} , not provably total in PA).

Each increment of A 's first argument adds one iteration level — successor, addition, multiplication, exponentiation, tetration — and it is precisely this unbounded ascent through the exhaustion scale (the Ackermann rows, equivalently the finite fast-growing hierarchy), still *within* it, that places A on the boundary between what a Turing machine can compute with provably bounded effort and what it can compute at all.

(The low-level closed forms and the closing inequalities of §5.2 were machine-checked on all materializable instances; the theorems themselves are proved by induction, not by computation.)

References

- **Ackermann, J.** (1928). *Zum Hilbertschen Aufbau der reellen Zahlen*. *Mathematische Annalen* **107**, 459–488.
- **Péter, R.** (1956–58). *Ein kombinatorischer Satz über die Funktionenklasse der primitiv-rekursiven Funktionen*. *Acta Physico-Mathematica* **6**, 1–10; see also *Die Theorie der rekursiven Funktionen* (Akadémiai Kiadó, 1956).
- **Goodstein, R. L.** (1947). *On the computable functions*. *Journal of Symbolic Logic* **12**, 127–131.
- **Kleene, S. C.** (1936). *General form of recursive function*. *American Journal of Mathematics* **57**, 254–263.
- **Church, A.** (1936). *An unsolvability problem in arithmetic*. *Journal of the London Mathematical Society* **11**, 218–228.
- **Turing, A. M.** (1936). *On computable numbers, with an application to the Entscheidungsproblem*. *Proceedings of the London Mathematical Society* **42**, 230–265.
- **Rogers, H.** (1967). *Theory of Recursive Functions and Effective Computability*. MIT Press.
- **Cutland, N. J.** (1980). *Computability: An Introduction to Recursive Function Theory*. Cambridge University Press.
- **Schwartz, D. G.** (2025). *A Characterization of Turing Machines that Compute Primitive Recursive Functions*. arXiv:2510.18283 [cs.FL].
- **Hájek, P., & Pudlák, P.** (1993). *Metamathematics of First-Order Arithmetic*. Cambridge University Press. (Provably-total functions of PA; ordinal ε_0 ; majorization by the fast-growing hierarchy.)
- **Girard, J.-Y.** (1987). *Proof Theory and Logical Complexity*. (Ordinal analysis; provable recursion in PA.)
- **Kirby, L., & Paris, J.** (1982). *Accessible independence results for Peano arithmetic*. *Bulletin of the London Mathematical Society* **14**(4), 285–293. (The hydra game; f_{ε_0} .)
- **Odifreddi, P. E.** (1992). *Classical Recursion Theory*. North-Holland.